

Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
In-Class Exercise (11/14)

1. Consider your program running on a **parallel processor**. If your program is 75% parallelizable, what would be the **speedup** of your entire program when executed on a multiprocessor with **infinitely many cores**?

Answer:

$$1 / (0.25 + 0.75 / \infty) = 4x \text{ speedup}$$

2. Consider four threads all running on one **superscalar** R10K-like processor:

```
# thread A
A1: add $t1, $s5, $t1
A2: or  $t1, $t1, $s0
A3: addi $t6, $t1, 4
A4: beq $t6, $zero, A1
...

# thread B
B1: addi $s1, $s1, 4
B2: and  $s2, $s1, $t3
B3: sub  $s3, $t0, $s2
B4: bne  $s3, $s4, B1
...

# thread C
C1: and  $t0, $t1, $s0
C2: sub  $t3, $s4, $t0
C3: slt  $t4, $t3, $s2
C4: bne  $t4, $zero, C1
...

# thread D
D1: and  $s0, $t0, $t1
D2: or   $t0, $t5, $s0
D3: addi $t7, $t0, 4
D4: beq  $t7, $zero, D1
...
```

The processor is **four-wide superscalar** and can fetch (F), decode (De), dispatch (Di) and retire (R) up to four instructions in any given cycle. Assume that all instructions (add, addi, and, beq, bne, or, slt, sub) use the **same** type of reservation station and functional unit and take **one cycle** in the X stage. Assume that the threads' loops run indefinitely and that the branch instructions are always taken and predicted perfectly in the F stage.

- a) You enable **fine-grained context-switching**. In the first cycle, the processor fetches four instructions from thread A, then four instructions from thread B in the next cycle, then four from threads C and D, then back to thread A, and so on. The pipeline diagram for the first four loop iterations of all threads is shown on the next page, assuming **infinite hardware resources**. Note that instructions are still retired in the order that they are fetched, even if they are from different threads.

- i. What is the **minimum number of functional units** that this processor needs such that no instruction in this program ever needs to wait for a functional unit?

Answer:

4 functional units

- ii. What is the **minimum number of write ports** needed in the physical register file for this program?

Answer:

3 write ports = 4 instructions in C – 1 branch instruction

- iii. What is the **minimum number of reorder buffer entries** that this processor needs such that no instruction in this program ever needs to stall for a reorder buffer entry?

Answer:

37 reorder buffer entries

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
A1	F	De	Di	S	X	C	R																	
A2	F	De	Di	S	S	S	X	C	R															
A3	F	De	Di	S	S	S	S	S	X	C	R													
A4	F	De	Di	S	S	S	S	S	S	S	X	C	R											
B1		F	De	Di	S	X	C	R	R	R	R	R	R											
B2		F	De	Di	S	S	S	X	C	R	R	R	R											
B3		F	De	Di	S	S	S	S	S	X	C	R	R											
B4		F	De	Di	S	S	S	S	S	S	X	C	R											
C1			F	De	Di	S	X	C	R	R	R	R	R	R										
C2			F	De	Di	S	S	S	X	C	R	R	R	R										
C3			F	De	Di	S	S	S	S	S	X	C	R	R										
C4			F	De	Di	S	S	S	S	S	S	X	C	R										
D1			F	De	Di	S	S	X	C	R	R	R	R	R										
D2			F	De	Di	S	S	S	X	C	R	R	R	R										
D3			F	De	Di	S	S	S	S	S	X	C	R	R										
D4			F	De	Di	S	S	S	S	S	S	X	C	R										
A1			F	De	Di	S	X	C	R	R	R	R	R	R	R									
A2			F	De	Di	S	S	S	X	C	R	R	R	R	R									
A3			F	De	Di	S	S	S	S	S	X	C	R	R										
A4			F	De	Di	S	S	S	S	S	S	X	C	R										
B1				F	De	Di	S	X	C	R	R	R	R	R	R									
B2				F	De	Di	S	S	S	X	C	R	R	R	R									
B3				F	De	Di	S	S	S	S	X	C	R	R										
B4				F	De	Di	S	S	S	S	S	X	C	R										
C1					F	De	Di	S	X	C	R	R	R	R	R									
C2					F	De	Di	S	S	S	X	C	R	R	R									
C3					F	De	Di	S	S	S	S	X	C	R	R									
C4					F	De	Di	S	S	S	S	S	X	C	R									
D1						F	De	Di	S	X	C	R	R	R	R	R								
D2						F	De	Di	S	S	S	X	C	R	R	R								
D3						F	De	Di	S	S	S	S	X	C	R	R								
D4						F	De	Di	S	S	S	S	S	X	C	R								
A1							F	De	Di	S	X	C	R	R	R	R	R							
A2							F	De	Di	S	S	S	X	C	R	R	R							
A3							F	De	Di	S	S	S	S	S	X	C	R	R						
A4							F	De	Di	S	S	S	S	S	S	X	C	R						
B1								F	De	Di	S	X	C	R	R	R	R	R						
B2								F	De	Di	S	S	S	X	C	R	R	R						
B3								F	De	Di	S	S	S	S	S	X	C	R						
B4								F	De	Di	S	S	S	S	S	S	X	C	R					
C1									F	De	Di	S	X	C	R	R	R	R						
C2									F	De	Di	S	S	S	X	C	R	R						
C3									F	De	Di	S	S	S	S	S	X	C	R					
C4									F	De	Di	S	S	S	S	S	S	X	C	R				
D1										F	De	Di	S	X	C	R	R	R	R					
D2										F	De	Di	S	S	S	X	C	R	R					
D3										F	De	Di	S	S	S	S	X	C	R					
D4										F	De	Di	S	S	S	S	S	X	C	R				
A1											F	De	Di	S	X	C	R	R	R					
A2											F	De	Di	S	S	S	X	C	R					
A3											F	De	Di	S	S	S	S	X	C	R				
A4											F	De	Di	S	S	S	S	S	X	C	R			
B1												F	De	Di	S	X	C	R	R					
B2												F	De	Di	S	S	S	X	C	R				
B3												F	De	Di	S	S	S	S	X	C	R			
B4												F	De	Di	S	S	S	S	S	X	C	R		
C1													F	De	Di	S	X	C	R	R				
C2														F	De	Di	S	S	S	X	C	R		
C3															F	De	Di	S	S	S	S	X	C	
C4																F	De	Di	S	S	S	S	S	X
D1																	F	De	Di	S	X	C	R	R
D2																	F	De	Di	S	S	S	X	C
D3																		F	De	Di	S	S	S	S
D4																			F	De	Di	S	S	S

Answer:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
A1	F	De	Di	S	X	C	R																	
A2	F	De	Di	S	S	S	X	C	R															
A3	F	De	Di	S	S	S	S	X	C	R														
A4	F	De	Di	S	S	S	S	S	S	S	X	C	R											
B1		F	De	Di	S	X	C	R	R	R	R	R	R											
B2		F	De	Di	S	S	S	X	C	R	R	R	R											
B3		F	De	Di	S	S	S	S	S	X	C	R	R											
B4		F	De	Di	S	S	S	S	S	S	S	X	C	R										
C1			F	De	Di	S	X	C	R	R	R	R	R	R										
C2			F	De	Di	S	S	S	X	C	R	R	R	R										
C3			F	De	Di	S	S	S	S	S	X	C	R	R										
C4			F	De	Di	S	S	S	S	S	S	X	C	R										
D1				F	De	Di	S	X	C	R	R	R	R	R	R									
D2				F	De	Di	S	S	S	X	C	R	R	R	R									
D3				F	De	Di	S	S	S	S	S	X	C	R	R									
D4				F	De	Di	S	S	S	S	S	S	X	C	R									
A1					F	De	Di	S	X	C	R	R	R	R	R	R								
A2					F	De	Di	S	S	S	X	C	R	R	R	R								
A3					F	De	Di	S	S	S	S	X	C	R	R									
A4					F	De	Di	S	S	S	S	S	S	X	C	R								
B1						F	De	Di	S	X	C	R	R	R	R	R								
B2						F	De	Di	S	S	S	X	C	R	R	R								
B3						F	De	Di	S	S	S	S	X	C	R	R								
B4						F	De	Di	S	S	S	S	S	S	X	C	R							
C1							F	De	Di	S	X	C	R	R	R	R	R							
C2							F	De	Di	S	S	S	X	C	R	R	R							
C3							F	De	Di	S	S	S	S	X	C	R	R							
C4							F	De	Di	S	S	S	S	S	X	C	R							
D1								F	De	Di	S	X	C	R	R	R	R							
D2								F	De	Di	S	S	S	X	C	R	R	R						
D3								F	De	Di	S	S	S	S	X	C	R	R						
D4								F	De	Di	S	S	S	S	S	X	C	R						
A1									F	De	Di	S	X	C	R	R	R	R	R	R				
A2									F	De	Di	S	S	S	X	C	R	R	R	R				
A3									F	De	Di	S	S	S	S	X	C	R	R	R				
A4									F	De	Di	S	S	S	S	S	X	C	R	R				
B1										F	De	Di	S	X	C	R	R	R	R	R				
B2										F	De	Di	S	S	S	X	C	R	R	R				
B3										F	De	Di	S	S	S	S	X	C	R	R				
B4										F	De	Di	S	S	S	S	S	X	C	R				
C1											F	De	Di	S	X	C	R	R	R	R	R	R		
C2											F	De	Di	S	S	S	X	C	R	R	R	R		
C3											F	De	Di	S	S	S	S	X	C	R	R	R		
C4											F	De	Di	S	S	S	S	S	X	C	R	R		
D1												F	De	Di	S	X	C	R	R	R	R	R		
D2												F	De	Di	S	S	S	X	C	R	R	R		
D3												F	De	Di	S	S	S	S	X	C	R	R		
D4												F	De	Di	S	S	S	S	S	S	X	C	R	
A1													F	De	Di	S	X	C	R	R	R	R	R	
A2													F	De	Di	S	S	S	X	C	R	R	R	
A3													F	De	Di	S	S	S	S	X	C	R	R	
A4													F	De	Di	S	S	S	S	S	S	X	C	
B1														F	De	Di	S	X	C	R	R	R	R	
B2															F	De	Di	S	S	S	X	C	R	
B3																F	De	Di	S	S	S	S	X	C
B4																	F	De	Di	S	S	S	S	X
C1																	F	De	Di	S	X	C	R	R
C2																	F	De	Di	S	S	S	X	C
C3																	F	De	Di	S	S	S	S	X
C4																	F	De	Di	S	S	S	S	S
D1																		F	De	Di	S	X	C	R
D2																		F	De	Di	S	S	X	C
D3																		F	De	Di	S	S	S	X
D4																		F	De	Di	S	S	S	S

b) You enable **simultaneous multithreading** instead. In each cycle, the processor fetches one instruction each from thread A, B, C and D (in that order). The pipeline diagram for the first four loop iterations of all threads is shown on the next page, assuming **infinite hardware resources**. Note that instructions are still retired in the order that they are fetched, even if they are from different threads.

- i. What is the **minimum number of functional units** that this processor needs such that no instruction in this program ever needs to wait for a functional unit?

Answer:

8 functional units

- ii. What is the **minimum number of write ports** needed in the physical register file for this program?

Answer:

8 write ports

- iii. What is the **minimum number of reorder buffer entries** that this processor needs such that no instruction in this program ever needs to stall for a reorder buffer entry?

Answer:

28 reorder buffer entries

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
A1	F	De	Di	S	X	C	R																	
B1	F	De	Di	S	X	C	R																	
C1	F	De	Di	S	X	C	R																	
D1	F	De	Di	S	X	C	R																	
A2		F	De	Di	S	S	X	C	R															
B2		F	De	Di	S	S	X	C	R															
C2		F	De	Di	S	S	X	C	R															
D2		F	De	Di	S	S	X	C	R															
A3			F	De	Di	S	S	S	X	C	R													
B3			F	De	Di	S	S	S	X	C	R													
C3			F	De	Di	S	S	S	X	C	R													
D3			F	De	Di	S	S	S	X	C	R													
A4			F	De	Di	S	S	S	S	X	C	R												
B4			F	De	Di	S	S	S	S	X	C	R												
C4			F	De	Di	S	S	S	S	X	C	R												
D4			F	De	Di	S	S	S	S	X	C	R												
A1				F	De	Di	S	X	C	R	R	R	R	R										
B1				F	De	Di	S	X	C	R	R	R	R	R										
C1				F	De	Di	S	X	C	R	R	R	R	R										
D1				F	De	Di	S	X	C	R	R	R	R	R										
A2					F	De	Di	S	S	X	C	R	R	R										
B2					F	De	Di	S	S	X	C	R	R	R	R									
C2					F	De	Di	S	S	X	C	R	R	R	R									
D2					F	De	Di	S	S	X	C	R	R	R	R									
A3						F	De	Di	S	S	S	X	C	R	R									
B3						F	De	Di	S	S	S	X	C	R	R									
C3						F	De	Di	S	S	S	X	C	R	R									
D3						F	De	Di	S	S	S	X	C	R	R									
A4							F	De	Di	S	S	S	S	X	C	R								
B4							F	De	Di	S	S	S	S	X	C	R								
C4							F	De	Di	S	S	S	S	X	C	R								
D4							F	De	Di	S	S	S	S	X	C	R								
A1								F	De	Di	S	X	C	R	R	R	R							
B1								F	De	Di	S	X	C	R	R	R	R	R						
C1								F	De	Di	S	X	C	R	R	R	R	R						
D1								F	De	Di	S	X	C	R	R	R	R	R						
A2									F	De	Di	S	S	X	C	R	R	R	R					
B2									F	De	Di	S	S	X	C	R	R	R	R					
C2									F	De	Di	S	S	X	C	R	R	R	R					
D2									F	De	Di	S	S	X	C	R	R	R	R					
A3										F	De	Di	S	S	S	X	C	R	R					
B3										F	De	Di	S	S	S	X	C	R	R					
C3										F	De	Di	S	S	S	X	C	R	R					
D3										F	De	Di	S	S	S	X	C	R	R					
A4											F	De	Di	S	S	S	S	X	C	R				
B4											F	De	Di	S	S	S	S	X	C	R				
C4											F	De	Di	S	S	S	S	X	C	R				
D4											F	De	Di	S	S	S	S	X	C	R				
A1												F	De	Di	S	X	C	R	R	R	R			
B1												F	De	Di	S	X	C	R	R	R	R	R		
C1												F	De	Di	S	X	C	R	R	R	R	R		
D1												F	De	Di	S	X	C	R	R	R	R	R		
A2													F	De	Di	S	S	X	C	R	R	R	R	
B2													F	De	Di	S	S	X	C	R	R	R	R	
C2													F	De	Di	S	S	X	C	R	R	R	R	
D2													F	De	Di	S	S	X	C	R	R	R	R	
A3														F	De	Di	S	S	S	X	C	R	R	
B3														F	De	Di	S	S	S	X	C	R	R	
C3														F	De	Di	S	S	S	X	C	R	R	
D3														F	De	Di	S	S	S	X	C	R	R	
A4															F	De	Di	S	S	S	S	X	C	
B4															F	De	Di	S	S	S	S	X	C	
C4															F	De	Di	S	S	S	S	X	C	
D4															F	De	Di	S	S	S	S	X	C	

Answer:

[illegible]

- c) Which of the two implementations above (**fine-grained context-switching** vs. **simultaneous multithreading**) needs more **reservation stations** for this program? Briefly explain why this is the case.

Answer:

Fine-grained context-switching needs more reservation stations.

With this implementation, dependent instructions are fetched closer (in time) to each other, causing instructions to occupy reservation stations for a longer duration while waiting for their operands to be ready (up to 7 cycles, compared to up to 4 cycles with simultaneous multithreading).

- d) If there were **no register data dependences** in this program (i.e., the ideal case), what would be the **minimum number of reorder buffer entries** needed in this processor to ensure no stalling?

Answer:

At any given cycle, there would be 4 instructions in S, 4 in X, 4 in C and 4 in R. Thus 16 reorder buffer entries.